

```
In [ ]: #File handling in python allows to work with computer files/server, it provides certian built in functions
```

```
In [ ]: #read a file
```

```
In [12]: file = open("D:\\Data Engineering\\Python\\Data - 127.txt",'r') #r stand for read
contents = file.read()
print(contents)
```

Date: 12-01-2023

The 5 best practices that I follow when developing a Power BI Data Model

1) Always develop a Start Schema Data Model - have noticed that many inexperienced data professionals often work directly with a flat file or single table when creating their reports

While this may be sufficient for small amounts of sample data, the report may become slow and prone to out-of-memory errors as the amount of data increases

2) Delegate as much processing to the data source as possible - To make the most efficient use of memory in Power BI, it is best to implement any necessary changes to columns, such as adding new columns or making necessary transformations in the source system as much as possible

This will allow all available memory to be used for building visuals

3) Load only necessary columns into your data model - To optimize your Power BI report and save memory, it is best to remove any columns in the backend data that you know will not be used in the report

For example, if you have 20 columns in your data but only need to use 15 of them, you should remove the unused columns in the Power Query steps

4) Aggregate the data to the finest level of granularity needed for your report - If you include data in your report that is more granular than necessary, you will be unnecessarily processing a large amount of data in Power BI to aggregate it

To optimize the amount of data being processed, always aggregate the data to the finest level required for the report

5) Avoid using Many-to-Many relationships in your data model - Many-to-Many relationships can create confusion in the data and may lead to misleading results unless you have a thorough understanding of the nuances

```
In [ ]: # write to afile
```

```
In [10]: file = open("D:\\Data Engineering\\Python\\Data.txt",'w') # w stand for write
file.write("I am excited to announce that I participated in theData Hackathon, specifically focusing on Food Industry Market")
file.close()
```

```
In [ ]: #use cases
#1. in app, log files gets generated(file handling is very helpful reading the files)
#2. move the files from one location to another loocation
```

```
In [11]: import os          # os library
import shutil as s #(#shutil -- shell utility)--libray which works with desktop files
```

```
In [4]: s.move("D:\\Data Engineering\\Python\\source\\Data.txt","D:\\Data Engineering\\Python\\destination")
```

```
Out[4]: 'D:\\Data Engineering\\Python\\destination\\Data.txt'
```

```
In [5]: s.copy("D:\\Data Engineering\\Python\\source\\Data - 127.txt","D:\\Data Engineering\\Python\\destination")
```

```
Out[5]: 'D:\\Data Engineering\\Python\\destination\\Data - 127.txt'
```

```
In [7]: s.copy2("D:\\Data Engineering\\Python\\source\\Guidelines.txt","D:\\Data Engineering\\Python\\destination")
```

```
Out[7]: 'D:\\Data Engineering\\Python\\destination\\Guidelines.txt'
```

```
In [13]: s.copyfile("D:\\Data Engineering\\Python\\source\\Bike_Data.xlsx","D:\\Data Engineering\\Python\\destination\\Bike_Data1.xls")
```

```
Out[13]: 'D:\\Data Engineering\\Python\\destination\\Bike_Data1.xlsx'
```

```
In [14]: s.copyfile("D:\\Data Engineering\\Python\\source\\Guidelines12.txt", "D:\\Data Engineering\\Python\\destination\\Guidelines12.txt")
```

```
Out[14]: 'D:\\Data Engineering\\Python\\destination\\Guidelines12.txt'
```

```
In [23]: s.copystat("D:\\Data Engineering\\Python\\source\\Guidelines12.txt", "D:\\Data Engineering\\Python\\destination\\Guidelines12.txt")
```

```
-----  
FileNotFoundError                                Traceback (most recent call last)  
Cell In[23], line 1  
----> 1 s.copystat("D:\\Data Engineering\\Python\\source\\Guidelines12.txt", "D:\\Data Engineering\\Python\\destination\\Guidelines129.txt")  
  
File ~\anaconda3\Lib\shutil.py:376, in copystat(src, dst, follow_symlinks)  
    374     st = lookup("stat")(src, follow_symlinks=follow)  
    375     mode = stat.S_IMODE(st.st_mode)  
--> 376     lookup("utime")(dst, ns=(st.st_atime_ns, st.st_mtime_ns),  
    377                     follow_symlinks=follow)  
    378     # We must copy extended attributes before the file is (potentially)  
    379     # chmod()'ed read-only, otherwise setattr() will error with -EACCES.  
    380     _copyxattr(src, dst, follow_symlinks=follow)  
  
FileNotFoundError: [WinError 2] The system cannot find the file specified: 'D:\\Data Engineering\\Python\\destination\\Guidelines129.txt'
```

```
In [25]: src = "D:\\Data Engineering\\Python\\source\\Guidelines12.txt"  
dst = "D:\\Data Engineering\\Python\\destination\\Guidelines12.txt"  
s.copystat(src, dst)
```

```
In [29]: src = "D:\\Data Engineering\\Python\\source\\Guidelines15.txt"  
dst = "D:\\Data Engineering\\Python\\destination\\Guidelines15.txt"  
s.copystat(src, dst)
```

```
In [30]: print(s.copystat(src, dst))
```

None

```
In [31]: import getpass  
  
username = getpass.getuser()  
print("System Username:", username)
```

System Username: supar

```
In [32]: import os  
  
username = os.getlogin()  
print("System Username:", username)
```

System Username: supar

```
In [33]: import platform as p
```

```
In [35]: p.machine()
```

```
Out[35]: 'AMD64'
```

```
In [36]: p.processor()
```

```
Out[36]: 'Intel64 Family 6 Model 140 Stepping 2, GenuineIntel'
```

```
In [37]: p.uname()
```

```
Out[37]: uname_result(system='Windows', node='SUPARNABABU', release='10', version='10.0.22631', machine='AMD64')
```

```
In [38]: p.architecture()
```

```
Out[38]: ('64bit', 'WindowsPE')
```

In [40]: `p.system()`

Out[40]: 'Windows'

In []:

In []:

In []: